# Hierarchical Clustering — Complete Notes

## 1. What is Hierarchical Clustering?

**Hierarchical Clustering** is an **unsupervised machine learning algorithm** used to group similar data points into clusters.

It creates a **hierarchy of clusters**, which can be visualized using a **dendrogram**.

### Simple definition

> **Hierarchical Clustering groups similar data points by repeatedly merging or splitting clusters and represents the process using a dendrogram.**

---

# 2. Types of Hierarchical Clustering

There are two types:

### 1. Agglomerative Clustering ⭐

**Bottom → Up**

Starts with every data point as an individual cluster and repeatedly merges the closest clusters.

[A] [B] [C] [D] [E]
↓
[AB] [C] [D] [E]
↓
[AB] [CD] [E]
↓
[ABCD] [E]
↓
[ABCDE]

This is the **most commonly used type**.

### 2. Divisive Clustering

**Top → Down**

Starts with all data points in one cluster and repeatedly splits them.

[ABCDE]
↓
[ABC] [DE]
↓
[AB] [C] [DE]

For practical machine learning, you'll mostly encounter **Agglomerative Clustering**.

---

# 3. How Agglomerative Clustering Works

Suppose we have 5 data points:

A B C D E

### Step 1 — Every point is its own cluster

[A] [B] [C] [D] [E]

So initially:

**Number of clusters = number of data points**

---

### Step 2 — Find the closest clusters

Suppose A and B are closest:

[AB] [C] [D] [E]

---

### Step 3 — Continue merging

Suppose C and D are closest:

[AB] [CD] [E]

---

### Step 4

[ABCD] [E]

---

### Step 5

[ABCDE]

Finally, everything becomes **one cluster**.

### Important

Hierarchical clustering does **not** start with $K = 3$.

It starts with:

$N$ data points → $N$ clusters

and gradually goes:

$N \rightarrow \ldots \rightarrow 3 \rightarrow 2 \rightarrow 1$

---

# 4. Distance

To decide which points/clusters are close, we need a **distance measure**.

For example:

### Euclidean Distance

For two points:

$A = (2,3)$
$B = (5,7)$
$d=(5-2)2+(7-3)2d = \sqrt{(5-2)^2+(7-3)^2} \; d=5d = 5$

Smaller distance means **more similar/closer**.

---

# 5. Distance vs Linkage — Very Important

Don't confuse these two.

### Distance

Measures:

**Point → Point**

Example:

$A \leftrightarrow B$

"How far are A and B?"

Common distance metrics include:

- Euclidean
- Manhattan
- Cosine

### Linkage

Measures:

**Cluster → Cluster**

Example:

[AB] ↔ [CD]

"How should we calculate the distance between these two clusters?"

Common linkage methods:

- Single
- Complete
- Average
- Ward

---

# 6. Linkage Methods

## A. Single Linkage

Uses the **minimum distance** between two clusters.

Suppose:

Cluster 1 → A, B
Cluster 2 → C, D

Distances:

A-C = 5
A-D = 8
B-C = 3
B-D = 7

Single linkage:

min(5,8,3,7)=3\min(5,8,3,7)=3

So:

**Single linkage = distance between the closest pair of points.**

### Memory:

**Single → Smallest**

---

# 7. Complete Linkage

Uses the **maximum distance** between two clusters.

Using the same example:

5, 8, 3, 7

Complete linkage:

max⁡(5,8,3,7)=8\max(5,8,3,7)=8

So:

**Complete linkage = distance between the farthest pair of points.**

### Memory:

**Complete → Maximum**

---

# 8. Average Linkage

Calculates the **average of all pairwise distances**.

5, 8, 3, 7
5+8+3+74=5.75\frac{5+8+3+7}{4}=5.75

So:

**Average linkage = average distance between all pairs of points from the two clusters.**

### Memory:

**Average → Average**

---

# 9. Ward Linkage ⋆

Ward is different from the previous three.

It does not simply take minimum, maximum, or average distance.

Instead, it asks:

> **"If I merge these two clusters, how much will the within-cluster variance increase?"**

It chooses the merge that causes the **smallest increase in within-cluster variance**.

### Simple example

Suppose:

1 2 3 10 11

These naturally form:

Cluster 1 → 1,2,3
Cluster 2 → 10,11

Merging:

1 + 2

causes a small increase in variation.

But merging:

1 + 10

causes a much larger increase.

Therefore Ward prefers the first merge.

### Simple definition

> **Ward linkage merges the two clusters that result in the smallest increase in within-cluster variance.**

---

# 10. Does Ward stop when variance increases?

**No.**

This was an important point we discussed.

Ward always chooses the merge with the **smallest increase** in variance.

Eventually it must merge everything:

N clusters
↓
...
↓
3 clusters
↓
2 clusters
↓
1 cluster

Even the final merge can cause a large increase in variance.

It doesn't stop.

---

# 11. Why Does Ward Create Compact Clusters?

Ward tries to minimize within-cluster variation.

Therefore it generally creates:



rather than clusters containing very spread-out points.

So:

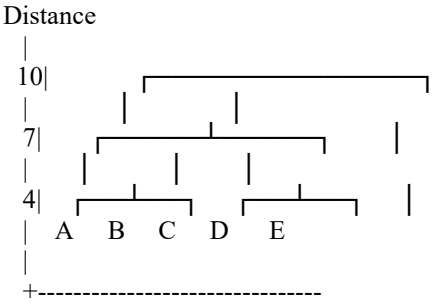
> **Ward tends to produce compact and relatively homogeneous clusters.**

---

# 12. Dendrogram

A **dendrogram** is a tree-like diagram that shows the sequence in which clusters are merged.

Example:




### Y-axis

Represents the **distance/dissimilarity at which clusters are merged**.

Higher merge height means the clusters being merged are more dissimilar.

---

# 13. How do we choose K using a Dendrogram?

This is very important.

We don't initially decide:

K = 3

Instead:

### Step 1

Create the hierarchical clustering.

### Step 2

Create the dendrogram.

### Step 3

Look for a **large vertical gap** between merging levels.

### Step 4

Draw an imaginary **horizontal line** through that gap.

### Step 5

Count the branches crossed by the line.

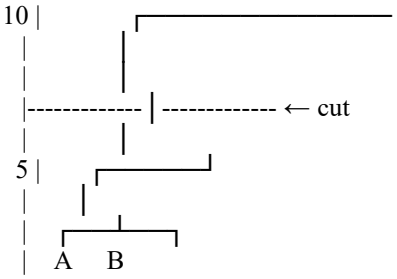
That gives the number of clusters.

---

## Example

Suppose the dendrogram suggests:

Distance




We place the cut in the large gap.

If the cut crosses **3 branches**, then:

K = 3

---

# 14. Why do we look for a large gap?

Suppose two clusters merge at:

Distance = 3

and the next merge happens at:

Distance = 10

That's a large jump.

It means the algorithm had to combine groups that were **much more dissimilar**.

Therefore, the groups below that gap may represent more natural clusters.

So:

> **Look for a large vertical gap and cut somewhere inside it.**

---

# 15. After choosing K

Suppose the dendrogram suggests:

K = 3

Now we can build the final Agglomerative Clustering model:

```
from sklearn.cluster import AgglomerativeClustering
model = AgglomerativeClustering(
   n_clusters=3,
   linkage='ward'
)
labels = model.fit_predict(X_scaled)
```

Now we get:

[0, 0, 1, 1, 2, 2, ...]

These are the cluster assignments.

---

# 16. Do we need Scaling?

Usually, **yes for numerical features**, because clustering is based on distances or variance.

Suppose:

| Age | Salary |
|---|---|
| 22 | 30000 |
| 25 | 50000 |
| 30 | 80000 |

Salary has much larger numerical values.

It can dominate the calculations.

So normally:

Raw Data
↓
Handle missing values
↓
Encode categorical variables
↓
Scale numerical features
↓
Hierarchical Clustering

For example:

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

---

# 17. Silhouette Score

After creating clusters, we want to know:

> **"How well are the data points assigned to their clusters?"**

That's where the **Silhouette Score** is useful.

It considers:

### 1. Cohesion

How close is a point to other points in its **own cluster**?

### 2. Separation

How far is the point from points in **other clusters**?

Ideally:

```
Same cluster       → close
Different clusters → far
```

---

# 18. Silhouette Score Range

The score ranges from:

$-1$ to $+1$-1 \text{ to } +1

Generally:

| Score | Interpretation |
|---|---|
| Close to +1 | Well-separated clusters |
| Around 0 | Clusters overlap |
| Negative | Some points may be assigned to the wrong cluster |

A **higher score generally indicates better-defined clusters**.

---

# 19. Calculate Silhouette Score

```
from sklearn.metrics import silhouette_score
score = silhouette_score(X_scaled, labels)
print(score)
```

Suppose:

```
Silhouette Score = 0.68
```

This suggests the resulting clusters have reasonably good separation and cohesion.

---

# 20. Important: What does Silhouette Score actually tell us?

Don't say:

❌ "Silhouette Score tells us the correct K."

Better:

> **Silhouette Score evaluates the quality of the clustering for a given K.**

For example, you could compare:

```
K = 2 → 0.45
K = 3 → 0.68
K = 4 → 0.52
```

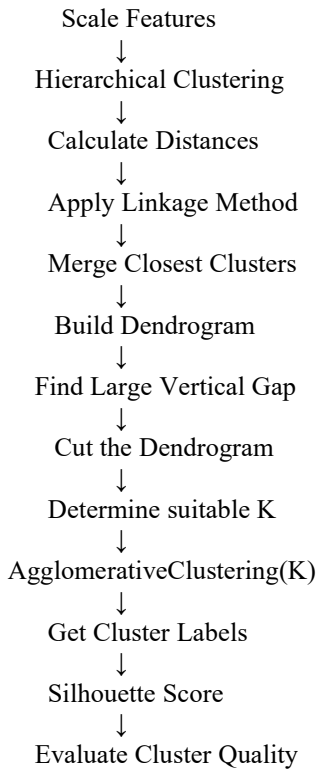
This gives you information about how well-defined the clustering is for each K.

---

# 21. Complete End-to-End Workflow ⭐

This is the complete process you should remember:

```
        DATA
         ↓
  Data Preprocessing
         ↓
Handle Missing Values
         ↓
Encode Categorical Data
         ↓
```




---

# 22. K-Means vs Hierarchical Clustering

| K-Means | Hierarchical |
|---|---|
| Need K before clustering | Build hierarchy first |
| Uses centroids | Uses distances/linkage |
| Iteratively updates centroids | Repeatedly merges clusters in agglomerative approach |
| No dendrogram | Uses dendrogram |
| Usually efficient for large datasets | Can be computationally expensive for large datasets |
| Final clusters directly produced | Hierarchy can be visualized |
| K is specified initially | K can be selected by cutting dendrogram |

---

# 23. Most Important Concepts to Remember

## Distance

Measures how far **two points** are from each other.

## Linkage

Defines how to measure the distance between **two clusters**.

### Single

**Minimum distance**

### Complete

**Maximum distance**

### Average

**Average distance**

### Ward

**Minimum increase in within-cluster variance**

### Dendrogram

**Shows the hierarchy/sequence of cluster merging.**

### Cut

**Choose a horizontal level on the dendrogram to determine the number of clusters.**

### Agglomerative Clustering

**Starts with individual points and repeatedly merges clusters.**

### Silhouette Score

**Evaluates how well-separated and cohesive the resulting clusters are.**

---

# ⭑ Final Memory Map

If you remember only this, you're good:

HIERARCHICAL CLUSTERING
↓
Start with N clusters
(one per data point)
↓
Calculate distance
↓

Linkage method
↓
Merge clusters
↓
Repeat
↓
Eventually 1 cluster
↓
DENDROGRAM
↓
Find large vertical gap
↓
Cut dendrogram
↓
Choose K
↓
Agglomerative Clustering
↓
Get cluster labels
↓
SILHOUETTE SCORE
↓
Evaluate cluster quality

## The 4 linkage methods in one line:

**Single → closest pair**
**Complete → farthest pair**
**Average → average pairwise distance**
**Ward → smallest increase in within-cluster variance**

This is the complete foundation you need for **Hierarchical Clustering**.